

Computational Thinking with Python

Session 3 – Functions

STUDENTS WILL LEARN TO CREATE AND USE FUNCTIONS

At the end of this session, you should be able to comprehend, trace, adapt and create Python code that:

- **Defines** a function using `def`
- **Calls** a function to execute its code
- Creates functions that use **parameters** (arguments)
- Uses **default argument values** for optional parameters
- **Returns** values from a function
- **Imports** and uses modules
- Handles **errors** gracefully with `try/except`

AGENDA

- 1 Function Basics
- 2 Parameters
- 3 Return Values
- 4 Modules
- 5 Error Handling

FUNCTIONS ELIMINATE CODE DUPLICATION AND IMPROVE ORGANIZATION

Problem:

Code in multiple places has drawbacks:

- Writing the same code twice
- Maintaining code in multiple places

Solution:

Functions help us:

- Reduce duplication – write once, use many times
- Organize code – break into smaller pieces
- Improve readability – meaningful names
- Simplify maintenance – fix bugs in one place

AGENDA

1 Function Basics

2 Parameters

3 Return Values

4 Modules

5 Error Handling

FUNCTIONS ARE NAMED SEQUENCES OF STATEMENTS THAT CAN BE REUSED

- A function is like a subprogram – a small program inside your program
- The basic idea: Write a sequence of statements and give it a name
- You can then execute this sequence anytime by **calling** the function

Two kinds of functions in Python:

1. **Built-in functions** – provided by Python (e.g., `print()`, `input()`, `int()`)
2. **User-defined functions** – functions we create ourselves



THE DEF KEYWORD STARTS A FUNCTION DEFINITION

```
def function_name():  
    code to execute
```

- `def` – Keyword that starts the function definition
- `function_name` – The name you give to your function
- `()` – Parentheses for parameters (empty if none)
- `:` – Colon is required after the parentheses
- Indented code – The function body must be indented

Important: Defining a function does not run it – you must call it!

PYTHON USES SNAKE_CASE FOR FUNCTION NAMES

Python naming conventions:

- Use lowercase letters
- Use underscores between words
- Choose descriptive names

snake_case is the Python convention!

Examples:

Good	Bad
<code>calculate_total</code>	<code>calculateTotal</code>
<code>print_receipt</code>	<code>pr</code> (too short)
<code>get_user_input</code>	<code>GetUserInput</code>

FUNCTIONS ARE DEFINED ONCE AND CALLED WHEN NEEDED

Step 1: Define the function

```
def happy_birthday():  
    print("Happy birthday!")  
    print("Happy birthday!")  
    print("Dear Homer!")  
    print("Happy birthday!")
```

Step 2: Call the function

```
happy_birthday()
```

The process:

1. Define the function with def
2. Write code inside (indented)
3. Call the function when needed

Output:

```
Happy birthday!  
Happy birthday!  
Dear Homer!  
Happy birthday!
```

BEGINNERS OFTEN FORGET PARENTHESES, COLONS, OR CALL ORDER

Common errors:

```
# Calling before defining
greet()      # Error!
def greet(): print("Hi")

# Forgetting parentheses
def greet(): print("Hi")
greet        # Does nothing!

# Forgetting the colon
def greet()  # SyntaxError!
```

Remember:

- Define the function **before** you call it
- Always use parentheses when calling:
func()
- Don't forget the colon after the definition
- Indent the function body (4 spaces)

Avoid these common mistakes!

PRACTICE FUNCTION BASICS WITH HANDS-ON EXERCISES

Practice: Open `01_Function_Basics` in your Codespace

Complete: `01_predict.py`, `02_predict.py`, `03_investigate.py`, `04_modify.py`

Predict → Investigate → Modify

AGENDA

1 Function Basics

2 **Parameters**

3 Return Values

4 Modules

5 Error Handling

ARGUMENTS ALLOW FUNCTIONS TO WORK WITH DIFFERENT DATA

Key concepts:

- Often we want to pass information to a function
- An **argument** is a value we pass into the function
- Arguments allow the function to do different things each time
- We define **parameters** in the definition to receive arguments

Example:

```
def happy_birthday(name):  
    print("Happy birthday to  
you!")  
    print("Happy birthday to  
you!")  
    print(f"Happy birthday,  
{name}!")  
    print("Happy birthday to  
you!")  
happy_birthday("Homer")  
happy_birthday("Marge")
```

PARAMETERS RECEIVE VALUES, ARGUMENTS PROVIDE THEM

Parameter:

- Variable in the function **definition**
- Acts as a placeholder
- Defined when you *create* the function

Example: `def greet(name): ...`

Argument:

- Actual value passed when **calling**
- Fills in the parameter
- Provided when you *use* the function

Example: `greet("Homer")`

Key Takeaway: **Parameter** = variable in definition; **Argument** = value when calling

MULTIPLE PARAMETERS ARE SEPARATED BY COMMAS

Definition and call:

```
def greet(name, room):  
    print(f"Hi {name}, room  
    {room}.")  
greet("Homer", 107)
```

Output: Hi Homer, room 107.

Arguments are matched by position!

Common error:

```
greet("Homer")  
# TypeError: missing 1 required  
# positional argument: 'room'
```

Why this happens:

- Function expects 2 arguments
- Only 1 was provided

KEYWORD ARGUMENTS MAKE FUNCTION CALLS MORE READABLE

Positional vs keyword:

```
def greet(name, room):  
    print(f"Hi {name}, room  
    {room}.")  
  
# Positional (order matters):  
greet("Homer", 107)  
  
# Keyword (order flexible):  
greet(room=107, name="Homer")
```

Benefits of keyword arguments:

- Order doesn't matter
Arguments are matched by name
- More readable
Clear what each argument represents
- Useful for many parameters
Easier to understand the call

Use keywords for clarity!

DEFAULT VALUES MAKE PARAMETERS OPTIONAL

Defining defaults:

```
def greet(name, msg="Good morning!"):
    print(f"Hello {name}, {msg}")

greet("Homer")
greet("Homer", "How are you?")
```

Output:

```
Hello Homer, Good morning!
Hello Homer, How are you?
```

How it works:

- If argument is provided: use that value
- If argument is omitted: use the default

Rule:

Parameters with defaults must come after those without:

```
# Wrong!
def greet(msg="Hi", name): ...
```

PRACTICE PARAMETERS WITH HANDS-ON EXERCISES

Practice: Open 02_Parameters in your Codespace

Complete: 01_predict.py, 02_predict.py, 03_investigate.py, 04_modify.py

Predict → Investigate → Modify

AGENDA

- 1 Function Basics
- 2 Parameters
- 3 Return Values**
- 4 Modules
- 5 Error Handling

THE RETURN STATEMENT SENDS DATA BACK TO THE CALLER

Key concepts:

- Functions can send data back
- `return` ends the function and sends a result
- Store the returned value in a variable

```
def area(w, h):  
    return w * h  
  
result = area(10, 5)  
print(result) # 50
```

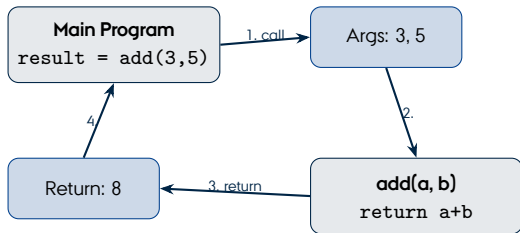
Key points:

- `return` sends a value back
- Function exits immediately
- Store the result to use it later

return exits the function!

FUNCTION CALLS PASS ARGUMENTS IN AND RETURN VALUES OUT

Visual flow:



The process:

1. Call function with arguments (3, 5)
2. Function receives values as (a, b)
3. Function calculates and returns result
4. Caller receives the return value (8)

RETURN VALUES CAN BE REUSED, PRINT OUTPUT CANNOT

Using print():

```
def add_print(a, b):  
    print(a + b)  
  
result = add_print(3, 4)  
print(result)
```

Output:

7

None

The value is displayed but not usable!

Using return:

```
def add_return(a, b):  
    return a + b  
  
result = add_return(3, 4)  
print(result)
```

Output:

7

The value is returned and can be used!

FUNCTIONS CAN CALL OTHER FUNCTIONS FOR COMPLEX CALCULATIONS

Tax calculation example:

```
def calculate_tax(amount,
rate=0.19):
    tax = amount * rate
    return tax

def calculate_total(price):
    tax = calculate_tax(price)
    return price + tax

price = 100
total = calculate_total(price)
print(f"Total: {total} EUR")
```

Output:

Total: 119.0 EUR

Key concepts:

- Functions can call other functions
- Default parameter (rate=0.19) for German VAT
- Return values enable composition

LOCAL VARIABLES EXIST ONLY INSIDE THEIR FUNCTION

Local variables:

- Created inside a function
- Only accessible within that function
- Destroyed when function ends

```
def my_func():  
    x = 10  # Local variable  
    print(x)  
  
my_func() # Output: 10  
print(x)  # Error! x not defined
```

Global variables:

- Created outside all functions
- Accessible from anywhere
- Exist for the entire program

```
y = 20    # Global variable  
  
def my_func():  
    print(y) # Can access global  
  
my_func() # Output: 20  
print(y)  # Output: 20
```


PRACTICE RETURN VALUES WITH HANDS-ON EXERCISES

Practice: Open `03_Return_Values` in your Codespace

Complete: `01_predict.py`, `02_predict.py`, `03_investigate.py`, `04_modify.py`

Predict → Investigate → Modify

AGENDA

- 1 Function Basics
- 2 Parameters
- 3 Return Values
- 4 Modules**
- 5 Error Handling

MODULES ADD EXTRA FUNCTIONS TO PYTHON

- Python has a variety of built-in functions: `print()`, `int()`, `input()`, ...
- For more functionality, we can use **modules** – collections of functions
- Use the `import` statement to load a module

```
import random  
for i in range(5):  
    print(random.randint(1, 25))
```

Usage pattern: `module_name.function_name()`

PYTHON INCLUDES MANY USEFUL STANDARD MODULES

random – Random numbers

```
import random
random.randint(1, 10)
random.choice(["a", "b", "c"])
```

math – Mathematical functions

```
import math
math.sqrt(16) # 4.0
math.pi      # 3.14159...
```

time – Time-related functions

```
import time
time.sleep(1) # Pause 1 second
```

datetime – Dates and times

```
from datetime import date
today = date.today()
print(today) # 2025-01-31
```

PYTHON OFFERS SEVERAL WAYS TO IMPORT MODULES

Syntax	Usage
<code>import random</code>	<code>random.randint(1, 10)</code>
<code>import random as rnd</code>	<code>rnd.randint(1, 10)</code>
<code>from random import randint</code>	<code>randint(1, 10)</code>
<code>from random import *</code>	<code>randint(1, 10)</code> (imports all)

Prefer `import module` or `from module import func`

PRACTICE MODULES WITH HANDS-ON EXERCISES

Practice: Open `04_Modules` in your Codespace

Complete: `01_predict.py`, `02_predict.py`, `03_investigate.py`, `04_modify.py`

Predict → Investigate → Modify

AGENDA

- 1 Function Basics
- 2 Parameters
- 3 Return Values
- 4 Modules
- 5 Error Handling**

ERROR HANDLING PREVENTS PROGRAMS FROM CRASHING

Problem:

Programs can crash unexpectedly:

- User enters text when number expected
- File doesn't exist
- Division by zero

```
age = int(input("Age: "))  
# User types "twenty" ->  
ValueError!
```

Solution:

Error handling allows you to:

- Catch errors before they crash
- Respond with helpful messages
- Continue running if possible

Don't let bad input crash programs!

THE TRY/EXCEPT STATEMENT CATCHES AND HANDLES ERRORS

`try:`

code that might fail

`except:`

code to run if error occurs

- `try` - Contains code that might cause an error
- `except` - Contains code to run if an error occurs
- If no error: `except` block is skipped
- If error occurs: `except` block runs, program continues

TRY/EXCEPT ALLOWS GRACEFUL ERROR RECOVERY

Without error handling:

```
age = int(input("Age: "))  
print(f"You are {age} years old")
```

If user types "twenty":

```
ValueError: invalid literal  
for int() with base 10: 'twenty'
```

Program crashes!

With error handling:

```
try:  
    age = int(input("Age: "))  
    print(f"You are {age}")  
except:  
    print("Please enter a  
number!")
```

If user types "twenty":

```
Please enter a number!
```

Program continues!

SPECIFIC ERROR TYPES ENABLE TARGETED ERROR MESSAGES

Multiple except blocks:

```
try:
    num = int(input("Number: "))
    result = 100 / num
    print(f"Result: {result}")
except ValueError:
    print("Not a valid number!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

Common error types:

- `ValueError` – Invalid value
- `ZeroDivisionError` – Division by zero
- `FileNotFoundError` – File missing
- `TypeError` – Wrong argument type
- `IndexError` – Index out of range

Catch specific errors for clarity!

INPUT VALIDATION COMBINES LOOPS WITH ERROR HANDLING

Robust input function:

```
def get_number(prompt):  
    while True:  
        try:  
            text = input(prompt)  
            return int(text)  
        except ValueError:  
            print("Invalid!")  
  
age = get_number("Age: ")
```

What this does:

1. Asks for input
2. Tries to convert to integer
3. If successful: returns the number
4. If error: prints message, loops again

This is called input validation!

PRACTICE ERROR HANDLING WITH HANDS-ON EXERCISES

Practice: Open `05_Error_Handling` in your Codespace

Complete: `01_predict.py`, `02_predict.py`, `03_investigate.py`, `04_modify.py`

Predict → Investigate → Modify

FUNCTIONS SHOULD BE SMALL, FOCUSED, AND WELL-NAMED

When to create a function:

- Organize your code into logical “paragraphs”
- Don’t repeat yourself – if you write the same code twice, make it a function
- Break down complexity – split large tasks into smaller functions

Good function design:

- Each function should do one thing well
- Use descriptive names that explain what the function does
- Keep functions short – if too long, split it up
- Use **return** values instead of `print()`

CLOSE THE SESSION BY BUILDING TWO COMPLETE PROGRAMS

Practice: Open 06_Make in your Codespace

Complete: 01_make.py, 02_make.py

Combine functions, parameters, returns, and error handling

FUNCTIONS MAKE CODE REUSABLE, ORGANIZED, AND MAINTAINABLE

- ✓ **Define** functions using `def` keyword
- ✓ **Call** functions by name with parentheses
- ✓ **Parameters** receive values; **arguments** pass values
- ✓ **Default values** make parameters optional
- ✓ **Return values** send data back with `return`
- ✓ **Modules** provide extra functions via `import`
- ✓ **Error handling** with `try/except` prevents crashes

Functions are the building blocks of good programs!

Thank You!

Nikolai Stein

